

Taller #4 IA - Reflexión y Análisis

Integrantes del equipo:

- Carolina Ceballos Gómez - 202510267
- Juan Felipe Leon - 202510397
- Miguel Aguilar - 202411646
- Andrés Sanabria - 202411507

Punto 1: Modelado del Dominio de Planificación

Tanto en el punto 1a como en el 1b no es posible hablar explícitamente de complejidades, pues estos action schemas y goals que fijamos, serán utilizados luego por otras funciones de planeación y su complejidad dependerá de con qué algoritmo se ejecuten.

Punto 2: Búsqueda hacia adelante (Forward Planning)

• Complejidad en espacio (Notación O): ¿Cuánta memoria necesita cada algoritmo en función del tamaño del espacio de estados (número de fluentes n, número de acciones a, profundidad del plan d)?

R/ La complejidad espacial es $O(b^d)$, siendo b el factor de ramificación y d la profundidad de la solución. Y en este problema b depende tanto del número de acciones posibles (a) como del número de fluentes (n). Por eso, a medida que aumenta el tamaño del mapa o la cantidad de pacientes o de supplies, el número de estados posibles también aumenta. Es decir, que BFS necesita bastante memoria cuando los mapas son grandes y completos.

• Complejidad en tiempo (Notación O): ¿Cuántos estados/nodos explora en el peor caso? • Completitud: ¿El algoritmo garantiza encontrar un plan si existe uno?

R/ La complejidad temporal es $O(b^d)$, siendo b el factor de ramificación y d la profundidad de la solución. En el peor caso, BFS explora todos los estados hasta la profundidad d, así que todos los nodos/estados. Sí, es completo, ya que si existe un plan válido, entonces BFS lo encontrará, pues recorre por niveles, pero además guarda los ya visitados para no entrar en ciclos infinitos.

• Optimalidad: ¿El algoritmo garantiza encontrar el plan de menor costo? ¿Bajo qué condiciones?

R/ BFS sí es óptimo siempre y cuando todas las acciones tengan el mismo costo, ya que el algoritmo encuentra el plan con el menor número de acciones, pero no revisa cuánto cuesta cada una, solo la cantidad de estas.

Layout	¿Encontró solución?	Longitud del plan	Estados expandidos	Tiempo de ejecución (s)	¿Óptimo?
cornerRescue	si	19	1470	0.503s	si
mediumRescue	si	32	5148	10.879s	si

warehouseRe cue	si	31	9716	39.227s	si
--------------------	----	----	------	---------	----

Punto 3: Búsqueda Regresiva (Backward Planning)

- Complejidad en espacio (Notación O): ¿Cuánta memoria necesita cada algoritmo en función del tamaño del espacio de estados (número de fluentes n , número de acciones a , profundidad del plan d)?

Necesita $O(b^d)$, por lo que mantiene en memoria la frontera de objetivos pendientes y los objetivos ya visitados para evitar ciclos. En el peor caso, cada objetivo regresado es un subconjunto distinto del espacio de fluentes, que tiene 2^n posibles subconjuntos (donde n es el número de fluentes). En la práctica, la frontera crece con el factor de ramificación b (número de acciones relevantes) a cada nivel de profundidad d , por lo que la complejidad en espacio es $O(b^d)$

- Complejidad en tiempo (Notación O): ¿Cuántos estados/nodos explora en el peor caso?

En el peor caso, el algoritmo explora todos los objetivos regresados posibles hasta profundidad d . Dado que en cada nivel se pueden aplicar hasta b acciones relevantes, el número total de nodos explorados es $O(b^d)$. Esto es lo mismo que forward BFS en teoría, pero en la práctica el factor de ramificación efectivo suele ser menor porque la regresión sólo considera acciones relevantes para el objetivo actual

- Completitud: ¿El algoritmo garantiza encontrar un plan si existe uno?

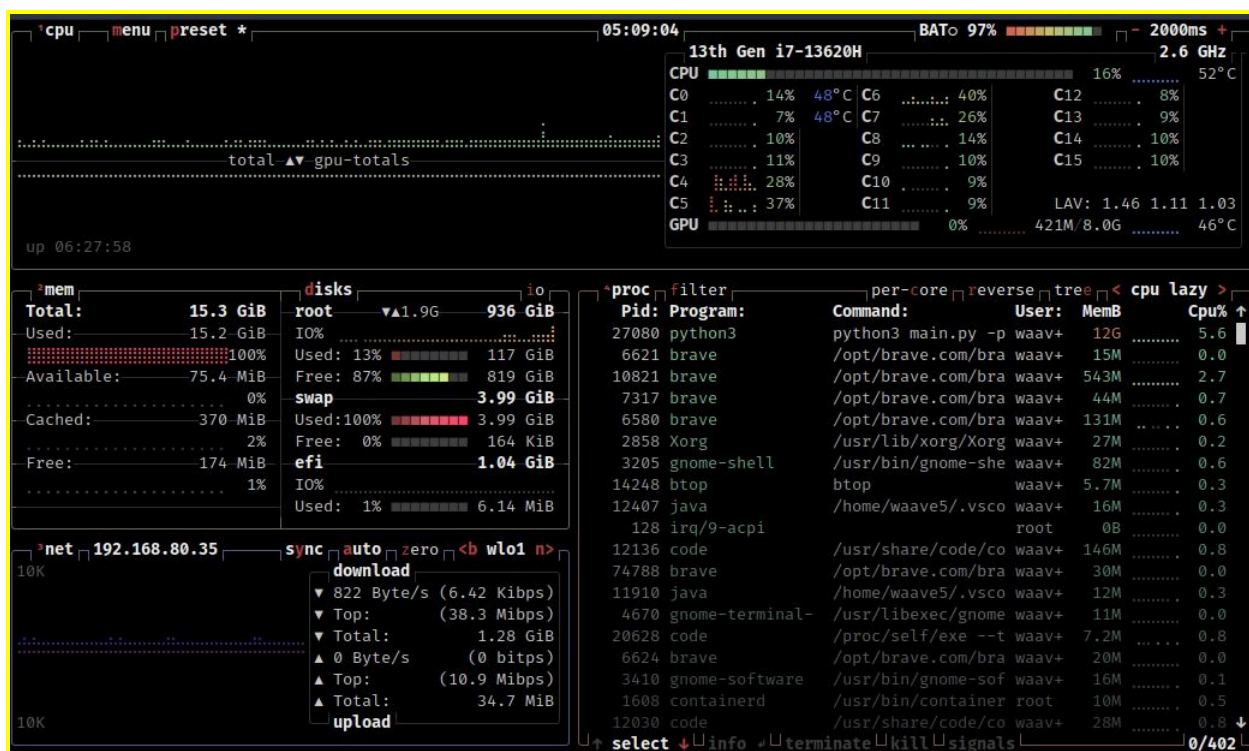
Sí, es completo, dado que el espacio de objetivos es finito y se usa una estructura de queue con detección de objetivos visitados, haciendo que el algoritmo garantice encontrar un plan si existe uno. Por lo que la condición clave es que el estado inicial sea alcanzable desde el objetivo regresando por acciones válidas.

- Optimalidad: ¿El algoritmo garantiza encontrar el plan de menor costo? ¿Bajo qué condiciones?

Sí es óptimo bajo la condición de que todas las acciones tengan el mismo costo unitario, como ocurre en este dominio de planificación. La búsqueda BFS explora los nodos por niveles de profundidad, por lo que el primer plan encontrado corresponde necesariamente al de menor número de acciones. En los experimentos realizados, los layouts tinyBase, tinyShift y tinyCross encontraron planes óptimos de longitud 9; tinyCorridor encontró un plan óptimo de longitud 10;

y tinySpiral un plan óptimo de longitud 11. Aunque todos los layouts fueron resueltos correctamente, el costo computacional aumentó drásticamente conforme crecía la complejidad estructural del mapa: tinyBase expandió 27.625 estados en 1,253 segundos, mientras que tinySpiral expandió 966.804 estados y tardó 708,745 segundos en completarse.

Adicionalmente, con los layouts originalmente propuestos en el taller, el algoritmo no lograba encontrar una solución en tiempos razonables debido al crecimiento exponencial del espacio de búsqueda y a las limitaciones de memoria del equipo utilizado. En una de las ejecuciones, el proceso asociado al programa permaneció ejecutándose durante aproximadamente 7 horas hasta que el sistema operativo terminó bloqueándolo por consumo excesivo de recursos. Por esta razón fue necesario diseñar layouts simplificados de tamaño 5×7 para poder analizar experimentalmente el comportamiento de backwardSearch y comparar métricas como longitud del plan, estados expandidos y tiempo de ejecución.



Layout	¿Encontró solución?	Longitud del plan	Estados expandidos	Tiempo de ejecución (s)	¿Óptimo?
tinyShift	Sí	9	71.509	6,124	Sí

tinyBase	Sí	9	27.625	1,253	Sí
tinyCross	Sí	9	39.323	2,140	Sí
tinyCorridor	Sí	10	276.775	73,283	Sí
tinySpiral	Sí	11	966.804	708,745	Sí

Punto 4: Heurísticas para Planificación

• Complejidad en espacio (Notación O): ¿Cuánta memoria necesita cada algoritmo en función del tamaño del espacio de estados (número de fluentes n, número de acciones a, profundidad del plan d)?

En ambas heurísticas se generan todas las acciones aterrizadas, entonces sabiendo que a es el número de acciones posibles, y n el número de fluentes, entonces el espacio $O(a*n)$, ya que la heurística almacena: todas las acciones aterrizadas y conjuntos de fluentes no satisfechos. Y cuando se utiliza junto con algoritmos de búsqueda como A*, el espacio total puede crecer hasta $O(b^d)$, donde b es el factor de ramificación y d la profundidad del plan, derivado del almacenamiento de nodos abiertos y cerrados.

• Complejidad en tiempo (Notación O): ¿Cuántos estados/nodos explora en el peor caso?

En ambas heurísticas la complejidad de tiempo es $O(a*n^2)$ ya que, en el peor caso, la heurística revisa repetidamente todas las acciones aterrizadas para encontrar aquella que cubra la mayor cantidad de metas no satisfechas y también como el estado relajado crece progresivamente .

No obstante, en la práctica la Heurística de ignorar listas de borrado suele explorar menos estados que la Heurística de ignorar precondiciones, ya que conserva las precondiciones y genera estimaciones más informativas del costo restante.

Cuando se integra con A*, el número total de estados explorados puede crecer exponencialmente respecto a la profundidad del plan, alcanzando un peor caso de $O(b^d)$.

• Optimalidad: ¿El algoritmo garantiza encontrar el plan de menor costo? ¿Bajo qué condiciones?

- Heurística de ignorar precondiciones: La heurística es admisible porque nunca sobreestima el costo real del plan: al ignorar precondiciones, asume un problema más

fácil que el original. Debido a esto, si se utiliza junto con A* y todas las acciones tienen costo no negativo, el algoritmo garantiza encontrar un plan óptimo. En consecuencia, las soluciones obtenidas con esta heurística mantienen garantías teóricas de optimalidad.

- Heurística de ignorar listas de borrado: También es admisible porque eliminando efectos negativos nunca hace que el problema relajado sea más difícil que el original. Sin embargo, esta implementación utiliza hill-climbing para resolver el problema relajado, seleccionando en cada paso la acción que cubre más metas pendientes. Debido a esto, la heurística no garantiza calcular exactamente el costo relajado mínimo y puede perder admisibilidad formal. Por tanto, aunque experimentalmente produjo planes de igual longitud y mejor desempeño, la optimalidad teórica de A* no queda completamente garantizada bajo esta implementación específica.

Layout	Heurística de ignorar precondiciones					Heurística de ignorar listas de borrado				
	¿Encontró solución?	Longitud del plan	Estados expandidos	Tiempo de ejecución (s)	¿Óptimo?	¿Encontró solución?	Longitud del plan	Estados expandidos	Tiempo de ejecución (s)	¿Óptimo?
cornerRescue	Sí	19	1638	33.670s	Sí	Sí	19	925	12.615s	Sí
mediumRescue	Sí	32	5581	680.541s	Sí	Sí	32	4390	255.358s	Sí
openRescue	Sí	26	9976	1594.11s	Sí	Sí	26	7120	431.177s	Sí
tinyBase	Sí	9	252	0.721s	Sí	Sí	9	138	0.28s	Sí
warehouseRescue	Sí	31	10320	3186.216s	Sí	Sí	31	8282	1152.316s	Sí

Punto 5: Planificación Jerárquica de Tareas (HTN)

• Complejidad en espacio (Notación O): ¿Cuánta memoria necesita cada algoritmo en función del tamaño del espacio de estados (número de flujos n, número de acciones a, profundidad del plan d)?

El algoritmo BFS tiene complejidad de $O(a^d)$ donde a es el número de acciones posibles por estado, esto porque debe almacenar todos los estados generados hasta la profundidad d. A* tiene una complejidad espacial de $O(n \cdot a^d)$ porque mantiene listas de nodos abiertos y cerrados, siendo n la cantidad de estados. El HTN requiere menos memoria porque trabaja con

refinamientos jerárquicos y no explora todos los estados posibles. Su complejidad depende del número de refinamientos r y la profundidad jerárquica h : $O(r^h)$

• Complejidad en tiempo (Notación O): ¿Cuántos estados/nodos explora en el peor caso? •
Complejidad: ¿El algoritmo garantiza encontrar un plan si existe uno?

BFS explora todos los nodos hasta encontrar la meta, por lo que su complejidad temporal también es exponencial: $O(b^d)$ A* tiene el mismo peor caso teórico, pero en la práctica explora menos estados gracias a las heurísticas. El HTN suele ser más rápido porque restringe la búsqueda mediante tareas abstractas y refinamientos ya definidos. Su costo aproximado es: $O((r^h) \cdot d)$

BFS es completo porque siempre encuentra una solución si existe y el espacio es finito. A* también es completo siempre que los costos sean positivos. El HTN no es completamente general, porque solo encuentra soluciones que estén representadas dentro de las jerarquías y refinamientos definidos en el código.

• Optimalidad: ¿El algoritmo garantiza encontrar el plan de menor costo? ¿Bajo qué condiciones?

BFS es óptimo cuando todas las acciones tienen el mismo costo, ya que encuentra el plan más corto. A* garantiza optimalidad si la heurística es admisible. El HTN no garantiza encontrar el plan óptimo porque usa refinamientos prefijados y acepta el primer plan válido encontrado.

• Para los Puntos 4 y 5: ¿Las heurísticas son admisibles (no sobreestiman)? ¿Son consistentes (satisfacen la desigualdad triangular)? ¿Cuál es la ventaja del planificador HTN sobre el planificador clásico en términos de complejidad?

Las heurísticas utilizadas en el Punto 4, como la distancia Manhattan o caminos mínimos, son admisibles porque no sobreestiman el costo real para llegar a la meta. Por ejemplo, la distancia Manhattan siempre calcula una distancia menor o igual al costo real en un grid. Además, estas heurísticas son consistentes porque satisfacen la desigualdad triangular. Esto significa que el costo estimado desde un nodo nunca es mayor que el costo de ir a un vecino más la estimación desde ese vecino.

La principal ventaja del planificador HTN frente al planificador clásico es que reduce significativamente la complejidad de búsqueda. Mientras los planificadores clásicos exploran muchas secuencias posibles de acciones, el HTN utiliza conocimiento del dominio mediante tareas abstractas y refinamientos ya definidos. Esto disminuye el branching factor efectivo,

reduce el número de estados explorados y mejora el tiempo de ejecución y el uso de memoria, especialmente en problemas grandes o complejos.

Layout	¿Encontró solución?	Longitud del plan	Estados expandidos	Tiempo de ejecución (s)	¿Óptimo?
TinyHTN	SI	9	0	0.000	Si
HTNBase	SI	12	0	0.001	Si
TinyMulti	SI	31	0	0.011	Si
DuoRescue	SI	28	0	0.010	Si
SmallMulti	SI	59	0	3.300	Si
CrossMulti	SI	37	0	0.002	Si

Reflexión sobre el uso de IA

Escriba una reflexión concisa sobre la co-construcción de la solución final con apoyo de la IA (vea política de uso de la IA más adelante), indicando, por ejemplo, qué aprendizajes obtuvo al ver las correcciones hechas por la IA, si éstas fueron básicas o solo mejoras secundarias; la facilidad o dificultad para crear el prompt adecuado, si la solución propuesta fue fácil de entender (¿usted la podría explicar en una sustentación?). Si no hizo ninguna iteración con IA, indique por qué no le pareció necesario ni útil. Aquí no hay respuestas correctas ni incorrectas. Este punto busca que usted se tome el tiempo de reflexionar sobre el uso de la IA en tareas de programación.

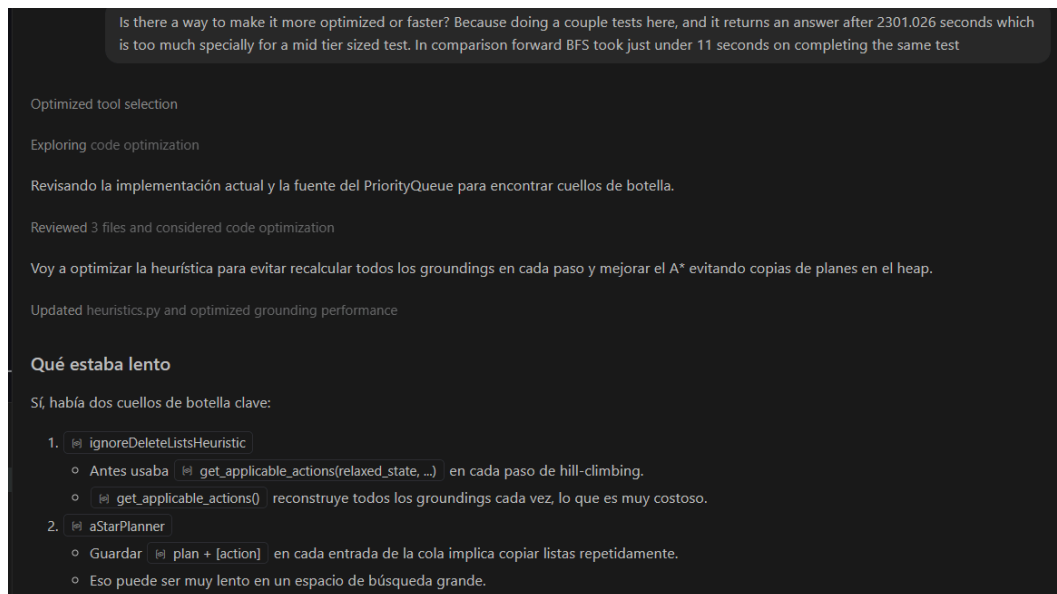
Este taller no se nos dificulta tanto como equipo, pues teníamos la mayoría de los conceptos claros (aunque algunos específicos sí le preguntamos a la IA) más que todo en el punto 3B, pero todo se resumió a la falta de capacidad de computo para una ejecución que expandía a millones de nodos en los mapas grandes; además de que con las pruebas era mucho más fácil verificar si lo que estábamos implementando estaba funcionando correctamente.

Así que no vimos que era necesario usar mucho la IA. Las únicas preguntas que le hicimos era si había manera de hacer reutilización de código pues había algunas funciones muy similares en las que cambiaban solo algunos detalles, aunque la IA nos respondía cosas valiosas y nos ayudó a expandir nuestro conocimiento de programación en python respecto a la reutilización de código, no implementamos los cambios o mejoras que nos arrojaba la respuesta pues nos parecía que nuestras implementaciones estaban bien y tal vez si las cambiamos para no rehusar, se pondrían

un poco confusas de entender ya que al tener tanta información nos arroja respuestas muy útiles, pero que por su necesidad de responder el prompt de la mejor forma que puede, no analizaba los cambios que sugería y que tal tal vez estos resultan aún más complejos o confusos en comparación con nuestra implementación actual. A excepción de la heurística de ignorar listas de borrado ya que la implementación que se realizó se demoraba demasiado en realizar las pruebas; por ejemplo para la prueba de mediumRescue, la implementación original se demoraba 2301 segundos mientras que con los arreglos, que eran fáciles de entender, se bajó a 255 segundos; y tenía errores que impedían una realización óptima la cual la IA detectó.

Pero principalmente se utilizó la IA para entender errores de lógica y en el caso del punto 3b, para ser consciente de agregar más podas para ir optimizando la solución, por lo que fue bastante útil acceder a la ayuda de la IA (Claude Opus 4.6) para comprender y realizar las correcciones pertinentes.

Conversaciones con Copilot de Visual Studio Code:



Qué se optimizó

heuristics.py

- Ahora usa `get_all_groundings(domain, objects)` una sola vez.
- En cada paso del hill-climbing solo filtra esos actions ya grounded por aplicabilidad.
- Esto elimina el costo de rehacer el grounding en cada iteración.

planner.py

- A* ahora empuja solo `(state, g)` en la cola de prioridad.
- El camino se reconstruye después desde `came_from`.
- Esto evita la creación de muchos planes parciales costosos.

Resultado esperado

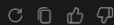
- `ignoreDeleteListsHeuristic` ya no repite la generación total de groundings tantas veces.
- `aStarPlanner` ya no copia planes en cada expansión.
- Debería ser significativamente más rápido para casos medianos/grandes.

Siguiente mejora posible

Si aún queda lento, el siguiente paso es:

- cachear la heurística por estado (`h_cache[state]`)
- y/o filtrar rápido las acciones que no contribuyen al objetivo actual antes de la comprobación de precondiciones.

Si quieres, puedo aplicar también el cacheo de heurística y la optimización adicional de acciones candidatas.



Raptor mini (Preview) • 1x

explicame, no hagas, como hacer hierarchiralSearch

Optimized tool selection

Read `htn.py`, lines 1 to 50

HierarchicalSearch es un algoritmo de planificación HTN (Hierarchical Task Network) que usa búsqueda en anchura (BFS) para refinar planes jerárquicos. Aquí va una explicación paso a paso de cómo implementarlo conceptualmente (sin código):

- Inicialización:** Comienza con un plan inicial que contiene una sola HLA (High-Level Action) de alto nivel, representando la tarea principal. Usa una cola (queue) para BFS, donde cada elemento es un plan parcial (lista de acciones/HLAs).
- Búsqueda en anchura:** Mientras la cola no esté vacía:
 - Toma el plan parcial del frente de la cola.
 - Verifica si el plan es completamente primitivo (solo acciones básicas) y satisface el estado objetivo del problema. Si sí, devuelve ese plan como solución.
 - Si no, encuentra la primera HLA no primitiva en el plan (de izquierda a derecha).
 - Para cada refinamiento posible de esa HLA (cada lista de subacciones/HLAs), genera nuevos planes parciales reemplazando la HLA con ese refinamiento.
 - Agrega estos nuevos planes a la cola.
- Refinamiento:** Cada refinamiento es una secuencia de acciones/HLAs que descompone la HLA. Asegúrate de que las acciones primitivas se apliquen correctamente al estado del problema (usando funciones como `is_applicable` y `apply_action`).
- Condiciones de parada:** Si encuentras un plan primitivo que alcanza el objetivo, detén. Si la cola se vacía sin solución, falla.
- Consideraciones:** Maneja estados del mundo para verificar aplicabilidad. Evita bucles infinitos limitando profundidad o usando heurísticas. En HTN, los

Backward Search

Expanded goals: 76553

Plan length: 9

Show more ▾

Eso te muestra clarísimo el problema: tu backward planner sigue expandiendo MUCHÍSIMOS subgoals.

Para `tinyBase` :

- solo 5x7
- plan de 9 acciones
- un único paciente
- un único supply

y aun así:

Expanded goals: 76553

Tiempo: 6.1s

Eso es enorme.

Comparación esperada

Forward BFS

Ya viste algo así:

Layout	Expandidos	Tiempo
tinyBase	~225	0.01s
smallRescue	~1181	0.27s

Comparación esperada

Forward BFS

Ya viste algo así:

Layout	Expandidos	Tiempo
tinyBase	~225	0.01s
smallRescue	~1181	0.27s

Eso es razonable.

Backward BFS ideal

Con buenas podas debería verse más parecido a:

Layout	Expandidos esperados	Tiempo esperado
tinyBase	500-5000	<1s
narrowRescue	5k-20k	1-5s
smallRescue	20k-100k	5-30s

Tu resultado actual

76553 goals

6s